



A Mini Project Report on “Mental Health Support Chatbot”

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Engineering

G H Patel College of Engineering & Technology

Submitted by

- 1. Mitali Sharma 12202040501040**
- 2. Naincy Rahuja 12202040501045**

Guided By

Dr. Zubin Bhaidasna

Academic Year (2024-2025)

**The Charutar Vidya Mandal (CVM) University,
Vallabh Vidyanagar - 388120**



G H Patel College of Engineering & Technology
Bachelor of Technology (Computer Engineering)

CERTIFICATE

This is to certify that **Mitali Sharma (12202040501040)** and **Naincy Rahuja (12202040501045)** has successfully submitted the **Mini Project Report** titled “**Mental Health Support Chatbot**”, in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Engineering** at **G H Patel College of Engineering & Technology** at **The Charutar Vidya Mandal (CVM) University**, **Vallabh Vidyanagar**, during the academic year **2024–2025**.

Dr. Zubin Bhaidasna
Internal Guide

Dr. Sudhir Vegad
Professor & HOD



DECLARATION

We, **Mitali Sharma (12202040501040)** and **Naincy Rahuja (12202040501045)**, hereby declare that this Mini Project report submitted in partial fulfillment for the degree of Bachelor of Technology in Computer Engineering, G H Patel College of Engineering & Technology. The Charutar Vidya Mandal (CVM) University, Vallabh Vidyanagar, is a bonafide record of work carried out by us under the supervision of Dr. Zubin Bhaidasna and that no part of this report has been directly copied from any student's reports or taken from any other source, without providing due reference.

Mitali Sharma

Naincy Rahuja

Acknowledgment

The completion of any work depends upon cooperation, co-ordination, and combined efforts of several sources of knowledge. We would like to express my deepest thanks to **Dr. Zubin Bhaidasna**, our mini project mentor for his valuable inputs, guidance, encouragement, and wholehearted cooperation throughout the duration of our mini project.

We would like to express our sincere thanks Head of the Department, **Dr. Sudhir Vegad** and Principal, **Dr. Kaushik Nath** for providing a chance for such a wonderful mini project opportunity from the institute.

We are also thankful to **Dr. Sudhir Vegad**, Head of the Department, and all faculty members of the **Department of Computer Engineering**, G H Patel College of Engineering and Technology, for providing a conducive learning environment and the necessary resources.

We extend our appreciation to CVM University, Vallabh Vidyanagar, for giving us this opportunity to work on a practical problem as a part of our academic curriculum.

Mitali Sharma
(12202040501040)

Naincy Rahuja
(12202040501045)

Abstract

The growing demand for accessible and affordable mental health care has motivated the development of this Mental Health Support Chatbot. With rising stress levels, anxiety, and depression, especially among youth, there is a critical need for non-judgmental, always-available platforms that offer emotional support and guidance. This project aims to bridge the gap between individuals in need and immediate mental health assistance using conversational AI. This project showcases how AI, when designed responsibly, can play a significant role in supporting mental well-being and reducing the stigma around mental health care.

LIST OF FIGURES

Fig 1.1: Overall flow diagram of project.....	2
Fig 4.1: Architecture diagram.....	9
Fig 4.2: Data flow diagram.....	11
Fig 4.3: Flowchart.....	12
Fig 4.4: Chatbot user interface.....	13
Fig 4.5: Chatbot soothing music feature.....	13

LIST OF TABLES

Table 2.1: Comparison between similar systems(chatbot).....	5
Table 3.1: Hardware requirements.....	7
Table 3.2: Software requirements.....	8
Table 4.1: Data flow in chatbot.....	10

TABLE OF CONTENTS

DECLARATION	i
ACKNOWLEDGEMENT.....	ii
ABSTRACT	iii
LIST OF FIGURES.....	iv
LIST OF TABLES.....	v
TABLE OF CONTENTS	vi
CHAPTER 1: INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Project Summary.....	1
1.3 Aim and Objective of Project.....	2
1.3.1 Aim of the Project.....	2
1.3.2 Objectives of the Project	2
CHAPTER 2: LITERATURE SURVEY	3
2.1 Overview	3
2.2 Review on similar systems	3
2.2.1 WOEBOT	3
2.2.2 WYSA	4
2.2.3 APPLE SIRI.....	4
CHAPTER 3: SYSTEM ANALYSIS	6
3.1 Introduction	6
3.2 Requirement Analysis	6
3.2.1 Functional Requirements.....	6
3.2.2 Non-Functional Requirements	6
3.3 Feasibility Study	6
3.4 Hardware and Software Requirements.....	7

3.4.1 Hardware Requirements.....	7
3.4.2 Software Requirements.....	8
CHAPTER 4: MODEL DESIGNING & UI	9
4.1 Architecture of the chatbot.....	9
4.2 Data Flow and Interaction Model.....	10
4.3 Flowchart	12
4.4 User Interface Design.....	13
CHAPTER 5: IMPLEMENTATION	14
5.1 Steps to develop the chatbot	14
5.2 Integration of Langchain and Gemini AI	15
5.3 Backend and Frontend Development	17
5.4 Testing and Debugging	18
CHAPTER 6: RESULT ANALYSIS	20
6.1 Performance Metrics.....	20
6.2 User feedback.....	22
CHAPTER 7: FUTURE WORK & CONCLUSION.....	24
7.1 Future Work	24
7.2 Conclusion.....	24
REFERENCES.....	25
Appendices.....	26
Appendix I.....	26

CHAPTER 1: INTRODUCTION

1.1 PROBLEM STATEMENT

Mental health issues such as anxiety, depression, and stress are becoming increasingly prevalent, yet access to professional psychological support remains limited due to factors like cost, stigma, and availability of mental health professionals. Many individuals struggle with emotional well-being but hesitate to seek help due to fear of judgment or lack of immediate support.

1.2 PROJECT SUMMARY

The AI-powered mental health chatbot is a conversational AI system designed to assist individuals facing emotional distress by offering empathetic conversations, coping strategies, and psychological resources. The chatbot is built using Gemini AI for NLP processing, LangChain for conversation management, and Retrieval-Augmented Generation (RAG) for enhancing response accuracy with external knowledge sources.

Key features of the chatbot include:

- Conversational AI offering empathetic, natural conversations for mental health support using advanced generative AI.
- Knowledge retrieval from credible mental health resources via RAG.
- Voice input support improving accessibility and comfort.
- Therapy music integration for relaxation and stress relief.
- Text-to-Speech (TTS) conversations enhancing engagement and support for visually impaired users.
- User-friendly UI for a clean, responsive and interactive user interface.
- Always accessible to users, ensuring support whenever needed.

1.3 AIM AND OBJECTIVE OF PROJECT

1.3.1 Aim of the Project

The primary aim of this project is to develop an AI-powered mental health support chatbot that provides empathetic, context-aware, and evidence-based guidance to individuals experiencing mental health challenges. The goal is to create a secure, accessible, and user-friendly chatbot that promotes emotional well-being while maintaining ethical and privacy standards.

1.3.2 Objectives of the Project

- Develop an AI-powered chatbot that can engage users in natural, empathetic conversations related to mental health concerns.
- Fine-tune Gemini AI on a dataset of user-psychologist conversations to improve response quality and relevance.
- Implement Retrieval-Augmented Generation (RAG) to fetch real-time, evidence-based mental health resources for enhanced response accuracy.
- Integrate LangChain for conversation memory and multi-turn dialogue management, ensuring context-aware responses.
- Ensure ethical AI implementation by incorporating bias mitigation, privacy measures, and user anonymity.

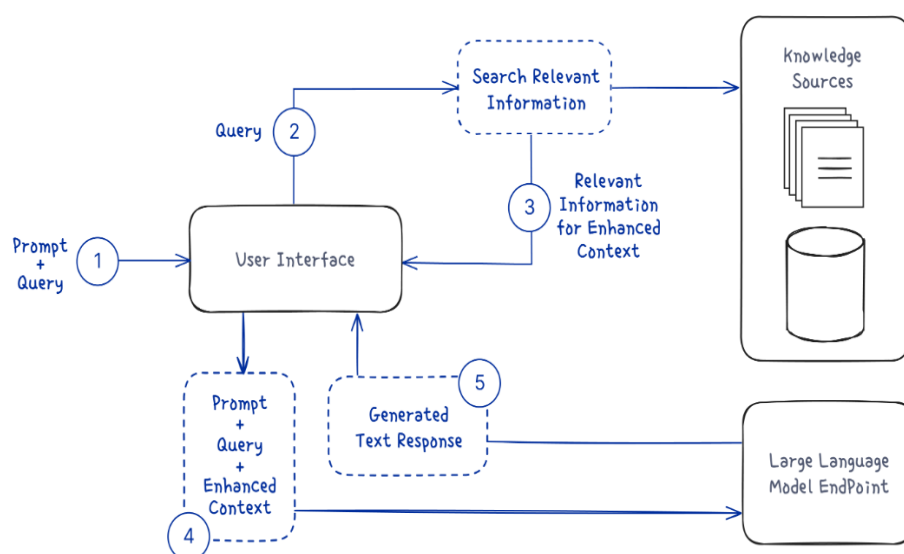


Fig 1.1 Overall Flow Diagram of Project

CHAPTER 2: LITERATURE SURVEY

2.1 OVERVIEW

The purpose of doing review on similar chatbots is to compare the chatbots that in the market and find out what are their strengths and weaknesses.

2.2 REVIEW ON SIMILAR SYSTEMS

2.2.1 WOEBOT

Introduction

Woebot is an artificially intelligent chatbot that uses the principles of cognitive-behavioral therapy (CBT) which is a researched clinical approach to treating depression. Woebot is a “fully automated conversational agent developed by Woebot Labs in San Francisco. It is designed to help people cope with feelings of depression and anxiety. It also can act as a tool to improve mental health.

Strengths

The conversation makes use of emoticons. Emoticons can help Woebot’s response look more emotional and human-like. It also set the tone of the conversation and make the conversation feel easier and relax. This help the users to easier to express their feelings to the chatbot.

Woebot do not randomly reply the users by searching answer on the internet. In fact, woebot answer the users by using the method called Cognitive Behavioral Therapy (CBT) which is an evidence-based approach to treatment.

Weakness

There are few limitations existed in Woebot .Woebot accept input by user typing words into the content box or select the option that they provided. In other words, Woebot do not provide oral communication with the user. This is inconvenient for those who have visual impairment.

2.2.2 WYSA

Introduction

Wysa, is an artificial intelligence based chatbot produced by an Indian startup called Touchkin. It uses evidence based therapies like Cognitive Behavior Therapy. Wysa helped many users to deal with depression, anxiety and suicidal thoughts. The conversations in the Wysa were private and encrypted.

Strengths

Wysa also use CBT method to help the user to restructuring negative thoughts. Wysa use not only one but two evidence based self-help techniques which is Cognitive Behavioral Therapy (CBT) and Dialectical behavior therapy (DBT) to life coaching the users.

Wysa will continue the conversation based on previous question. This make the conversation become more close to real life counseling example because all the conversation will focus on the feeling of the users. The conversation will not suddenly jump to other topic. In fact, it will stay in topic and direct the user on how to get over their difficulties.

Weaknesses

Wysa also has same limitations like Woebot which is accept input in the form of words or select the option provided. The interaction would be limited due to lack of sophisticated entry. With prefilled answers and guided journeys, Wysa felt more like an interactive quiz or game than a chatbot.

2.2.3 APPLE SIRI

Introduction

Siri (Speech Interpretation and Recognition Interface) is an artificial intelligent software built onto Apple's iOS and MacOS system. It uses Natural Language Processing (NLP) technology, allowing users to interact with their mobile phones to perform some function such as completing search, querying the weather, setting up a mobile calendar, setting alarms, and more (Apple,2018)

Strengths

Apple Siri able to answer all kinds of questions. If it encounter some questions that it unable to answer, it will perform searching on the internet. Apple Siri do not reply on internet to perform its task. It can still carry out some task such as make a phone call without internet.

Weaknesses

There are few weakness existed in Apple Siri. It required run on device which install iOS and MacOS only. In other words, other product which does not belong to Apple cannot use Siri. It does not build to deal with the emotional problem of the user. When it encounter any problem that out of its range, it only can reply the user by searching on the internet. The user can get the result that searching on the internet but not the comfort they want.

Table 2.1 Comparison between similar systems (chatbot)

Features/Name	Woebot	Wysa	Apple Siri
Conversational	Text-based	Text-based	Voice-based
Platform Supported	Cross Platform Apps	Cross Platform Apps	iOS & MacOS only
Cognitive behavioral therapy	Implemented	Implemented	Not Implemented
Memory	Reply based on previous conversation.	Reply based on previous conversation.	Reply not based on previous conversation.
Answer Specific pre-defined Question	Yes	Yes	No
Completely Free	No	No	Yes

CHAPTER 3: SYSTEM ANALYSIS

3.1 INTRODUCTION

System analysis is a crucial phase in the development of any software system, as it involves understanding user requirements, identifying technical feasibility, and selecting appropriate technologies. For the Mental Health Support Chatbot, system analysis focuses on evaluating functional and non-functional requirements, ensuring the system is practically feasible, and determining the necessary hardware and software resources for development and deployment.

3.2 REQUIREMENT ANALYSIS

3.2.1 Functional Requirements

- The system should accept text and voice input from the user.
- It must generate relevant and empathetic responses using a generative AI model.
- It should integrate therapy music suggestions through YouTube.
- It must provide a user-friendly interface for seamless interaction.
- RESTful API services should handle all client-server communication.

3.2.2 Non-Functional Requirements

- **Usability:** The interface should be simple and comforting.
- **Performance:** Responses should be delivered within 2 seconds.
- **Reliability:** The system should work smoothly with minimal downtime.
- **Scalability:** It should support multiple users without performance degradation.
- **Security:** All data transfers must be secured; no sensitive information should be stored without consent.

3.3 FEASIBILITY STUDY

3.3.1 Technical Feasibility

The chatbot utilizes technologies such as Flask, React, and Gemini AI, which are well-documented, easy to integrate, and supported by strong communities. These tools make the system technically viable and cost-effective.

3.3.2 Economic Feasibility

The project leverages free and open-source tools, reducing cost. Cloud services and APIs with free usage tiers help minimize hosting and maintenance expenses.

3.3.3 Operational Feasibility

The chatbot can be operated through a web browser, requiring no installation. Users only need a device with internet access and a microphone, making the solution practically accessible and operational.

3.4 HARDWARE AND SOFTWARE REQUIREMENTS

The successful implementation of the chatbot relies on meeting certain hardware and software requirements. These requirements ensure smooth performance, especially while handling real-time voice input and AI-generated responses.

3.4.1 Hardware Requirements

Table 3.1 outlines the recommended hardware specifications for developing and testing the chatbot system.

Table 3.1 Hardware requirements

Component	Specification
Processor	Intel i5 or equivalent and above
RAM	Minimum 8 GB
Storage	Minimum 256 GB HDD / SSD
Internet	Required for AI API (Gemini) & YouTube
Microphone	Required for voice input functionality
Graphics	Integrated graphics sufficient

3.4.2 Software Requirements

The required software stack includes frontend and backend technologies, programming tools, and third-party APIs. Table 3.2 lists all necessary software components.

Table 3.2 Software requirements

Software Component	Version / Details
Operating System	Windows 10/11, Linux (Ubuntu), or macOS
Python	3.9 or later
Flask	Web framework for RESTful APIs
React JS	Frontend framework
LangChain	For integrating generative AI
Gemini API	Google's generative AI model
Web Browser	Chrome / Firefox with Web Speech API
Postman / Insomnia	API testing tool
VS Code / PyCharm	IDE for development
Node.js + npm	For managing React dependencies
YouTube API	For therapy music integration

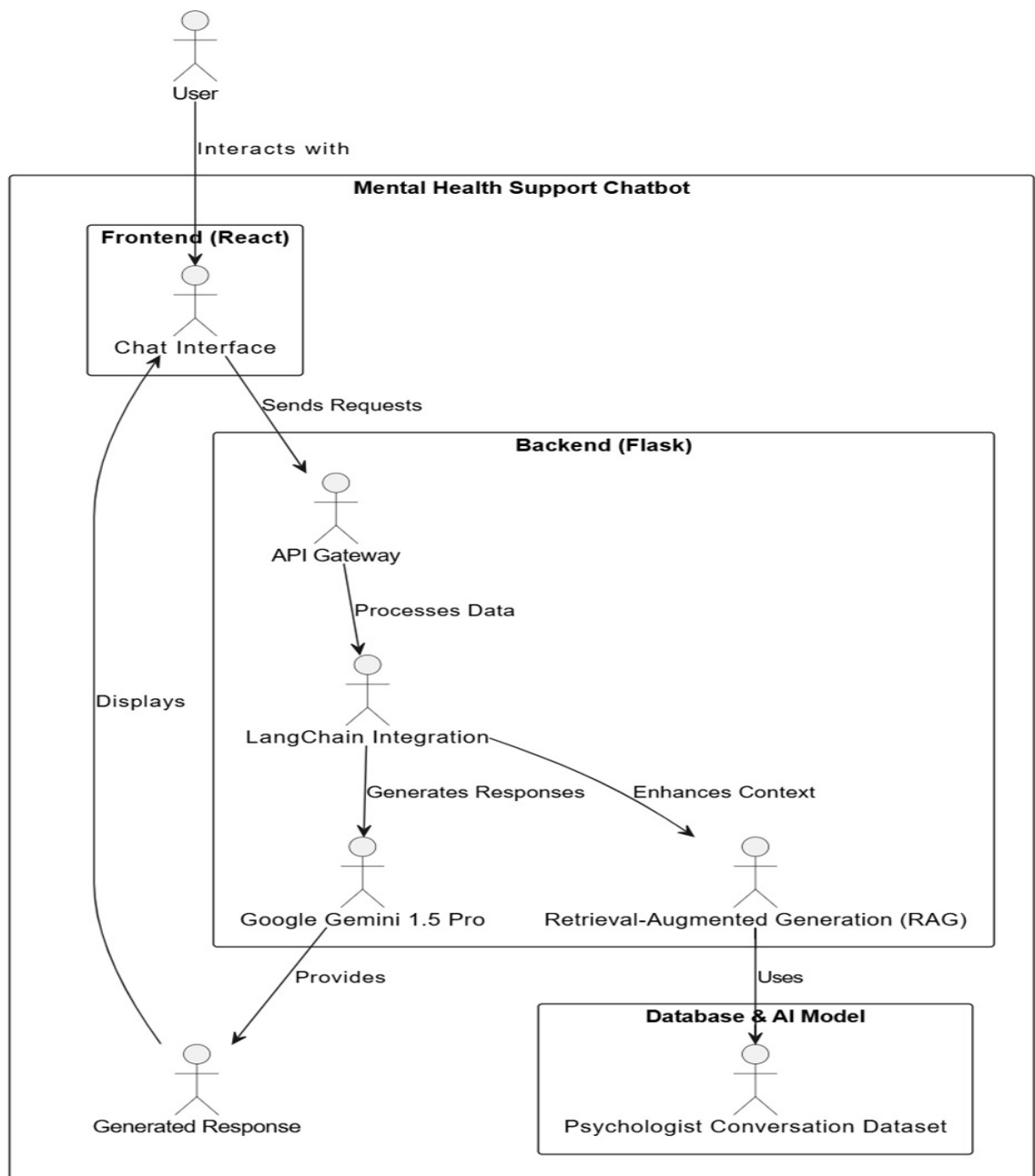
CHAPTER 4: MODEL DESIGNING & UI**4.1 ARCHITECTURE OF THE CHATBOT**

Fig 4.1 Architecture Diagram

4.2 DATA FLOW AND INTERACTION MODEL

The chatbot follows a structured data flow to process user inputs and generate appropriate responses.

1. **User Input:** The user sends a text or voice message through the UI.
2. **Preprocessing:** The input is tokenized and analyzed for intent detection and sentiment classification.
3. **Knowledge Retrieval (RAG):** If the chatbot lacks sufficient context, it fetches relevant information from external sources.
4. **Response Generation (Gemini AI):** The AI model generates a response based on the retrieved knowledge and user context.
5. **Context Management:** Conversation history is updated for maintaining contextual continuity.
6. **Output Delivery:** The chatbot provides a response via text or voice output.
7. **User Feedback & Logging:** The chatbot stores anonymized interaction logs to enhance future responses through continuous learning.

Table 4.1 Data Flow in Chatbot

Step	Description
User Input	User sends a message to the chatbot.
Processing in AI Model	Chatbot processes the input using NLP and sentiment analysis.
Generating Response	Response is generated based on the user's emotional state.
Providing Output	Chatbot provides feedback, resources, or escalates the situation if needed.

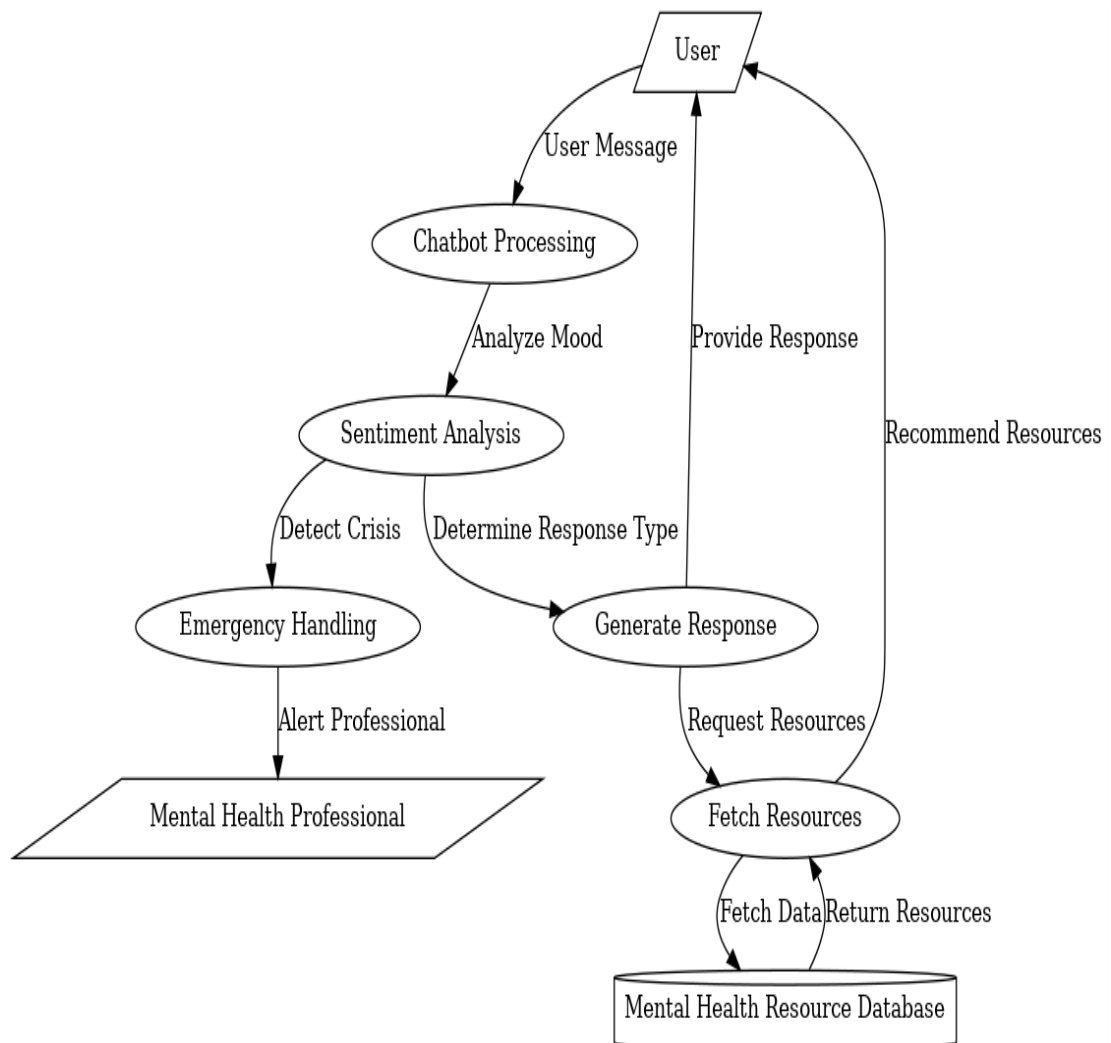


Fig 4.2 Data Flow Diagram

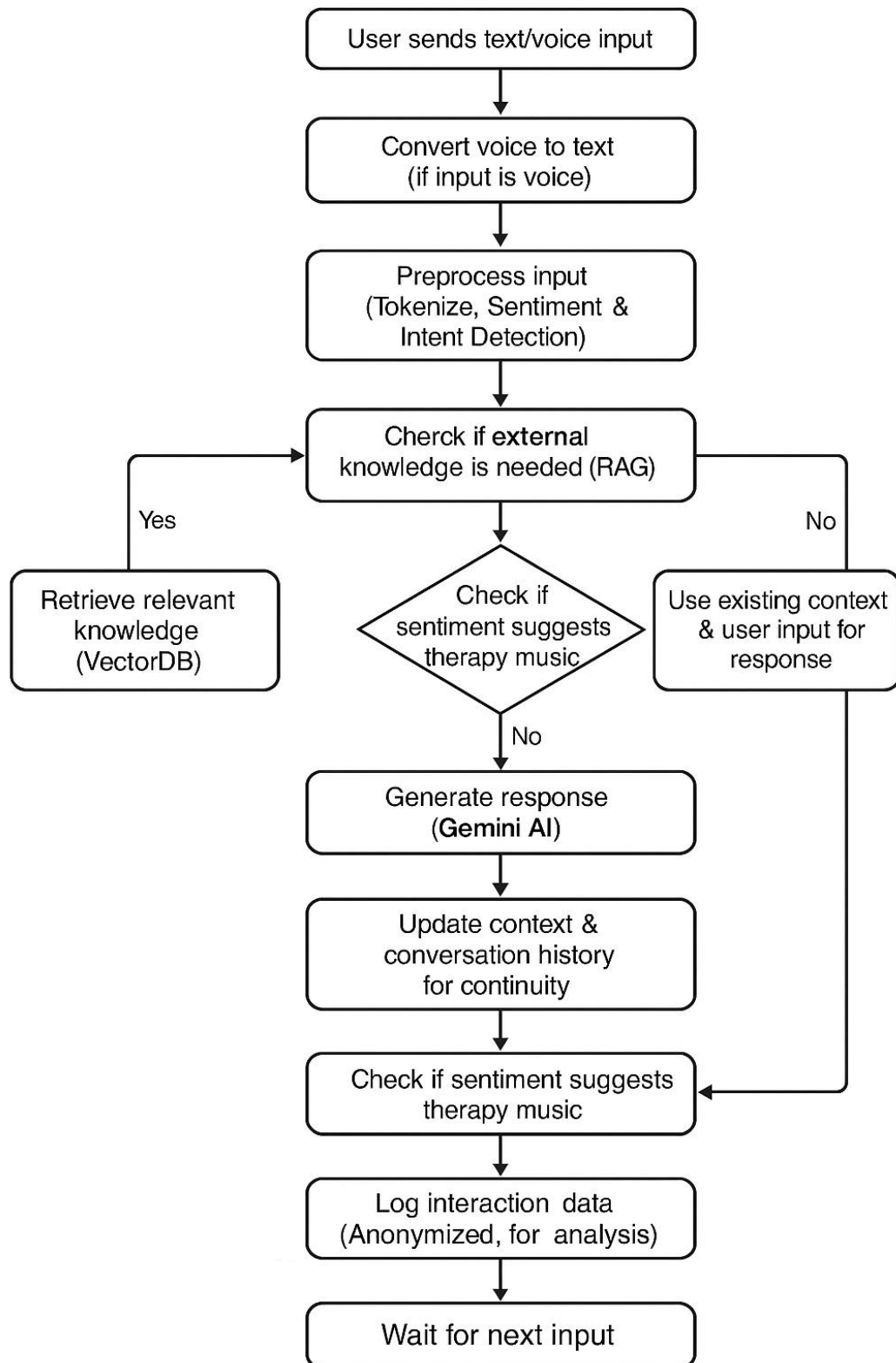
4.3 FLOWCHART

Fig 4.3 Flowchart

4.4 USER INTERFACE DESIGN

Chat Interface

- Simple, minimalistic UI with real-time response generation.
- Support for text and voice input.
- Emojis and icons to enhance expressiveness.

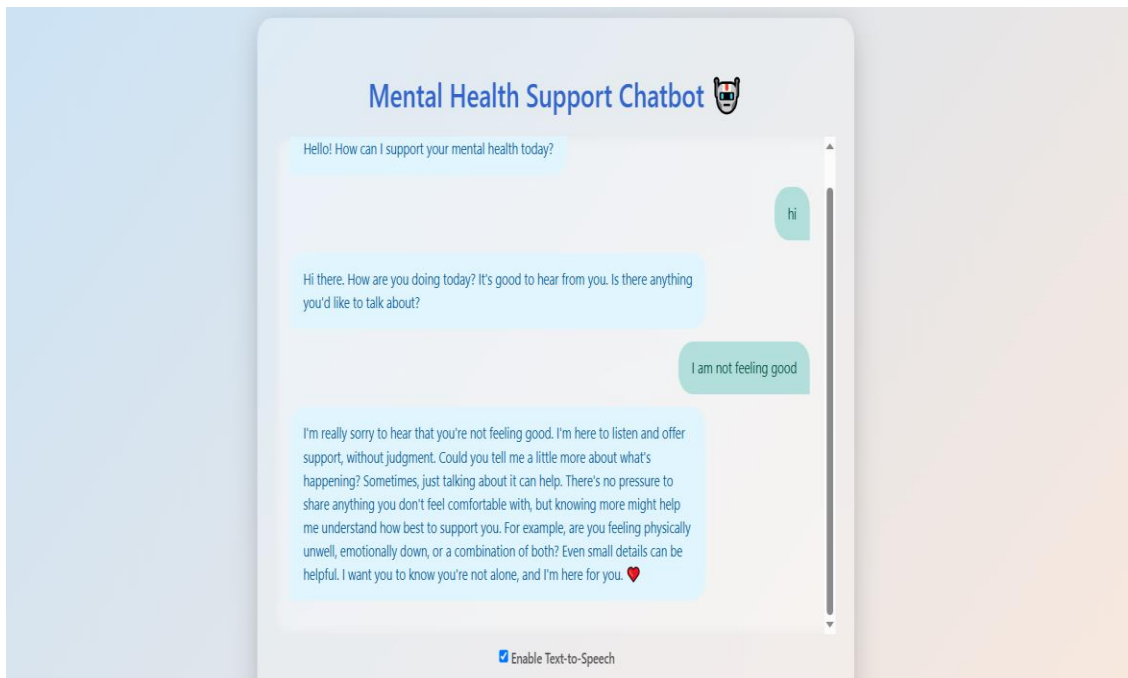


Fig 4.4 Chatbot User Interface

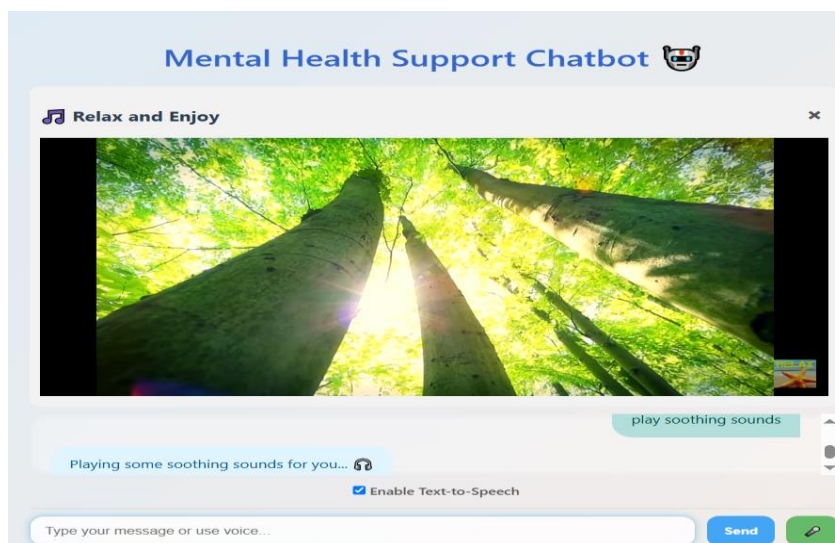


Fig 4.5 Chatbot Soothing Music Feature

CHAPTER 5: IMPLEMENTATION

5.1 STEPS TO DEVELOP THE CHATBOT

1. Project Setup

- Obtain access to the Gemini 1.5 Flash API by signing up on Google AI Studio and generating an API key.
- Set up a virtual environment and install the required libraries such as LangChain, Google Generative AI SDK, and ChromaDB for retrieval-based responses.

2. Define the Chatbot Workflow

- User Input Handling: Capture user queries related to mental health.
- System Prompt Definition: Design a well-structured system prompt to guide the chatbot's responses in a professional and empathetic manner.
- Response Generation: Combine retrieved context with the chatbot's generative abilities to provide accurate and meaningful responses.

3. Implement RAG for Enhanced Responses

- Collect verified mental health resources, conversation datasets, and psychologist-approved materials.
- Convert the text into vector embeddings and store them in a vector database (e.g., ChromaDB).
- Retrieve the most relevant documents for each query before passing them to the Gemini model, ensuring informed responses.

4. API Integration with Gemini 1.5 Flash

- Configure the API key and define parameters like temperature (to control response creativity) and max tokens (to limit response length).
- Implement the chatbot's backend logic to send user queries to Gemini 1.5 Flash and retrieve AI-generated responses.

5. System Prompt Engineering

- Develop a structured system prompt that defines the chatbot's tone, personality, and behavior.
- The system prompt should include:
 - Role: The chatbot acts as a mental health support assistant.
 - Guidelines: It should provide empathetic, non-judgmental, and informative responses.
 - Limitations: It should not provide medical diagnoses but encourage seeking professional help when needed.

6. Frontend and User Interface

- Design a user-friendly chatbot interface using frameworks like Flask.
- Ensure features like real-time chat, response delay simulation (to mimic human interaction), and soothing music.

7. Testing and Optimization

- Perform functional testing by evaluating different mental health queries and chatbot responses.
- Fine-tune the system prompt and RAG retrieval process to enhance accuracy and relevance.
- Implement safety checks to filter inappropriate or harmful responses.

8. Deployment and Maintenance

- Deploy the chatbot on a cloud server or web application for accessibility.
- Continuously update the knowledge base and refine the chatbot using user feedback.

5.2 INTEGRATION OF LANGCHAIN AND GEMINI AI

A. Setting Up LangChain

Configuring LangChain for Conversational Flow

- LangChain helps manage the structure of chatbot conversations. By integrating memory and retrieval mechanisms, it ensures responses feel continuous and context-aware.
- The chatbot stores user history (within session limits) to provide relevant responses while maintaining privacy.

Utilizing LangChain's Memory & Retrieval Modules

- Memory Module: Keeps track of user interactions within a session, allowing for context-aware responses.
- Retrieval Module: Uses a vector database to fetch previously discussed topics, ensuring personalized engagement.

B. Implementing Retrieval-Augmented Generation (RAG)

Storing Past Interactions for Personalized Responses

- Instead of relying solely on predefined responses, the chatbot uses RAG to pull in useful information from previous conversations and trusted resources.
- A vector database (e.g., FAISS or Pinecone) stores past interactions and retrieves relevant data when needed.

C. Enhancing Responses with Gemini AI

Using Prompt Engineering for Response Optimization

- Instead of fine-tuning (which Gemini AI doesn't support), prompts are carefully crafted to guide Gemini's responses toward empathetic and context-aware answers.
- Example: Instead of a generic "How are you feeling?", a dynamic prompt like "You mentioned feeling anxious earlier. Would you like to talk about it?" ensures continuity.

Generating Empathetic & Contextual Responses

- Gemini AI leverages its deep understanding of natural language to craft responses that are emotionally supportive and non-judgmental.

- It avoids generic replies and instead focuses on active listening techniques—acknowledging user emotions and offering suggestions tailored to their situation.

Enhancing Conversation Flow

- The chatbot blends retrieved responses from LangChain with AI-generated text from Gemini to create a seamless, human-like dialogue.
- Gemini AI ensures the chatbot does not provide medical advice, instead directing users to professional resources if necessary.

5.3 BACKEND AND FRONTEND DEVELOPMENT

Backend Development (Flask)

The backend in Flask will handle user interactions and AI processing.

A. Setting Up Flask Backend

1. Install Flask & Required Libraries

- Set up a virtual environment and install Flask, LangChain, and Gemini API SDK.

2. Define API Endpoints

- Create endpoints for handling user messages, fetching past interactions, and processing AI-generated responses.

3. Integrate LangChain & Gemini AI

- Process user inputs and generate responses using Gemini AI.

Frontend Development (React)

The React frontend will provide a seamless and user-friendly chat interface.

A. Building the User Interface**1. Set Up React Environment**

- Use Create React App for a fast development setup.

2. Develop the Chat UI

- Design an intuitive chatbot interface with a text input box and dynamic chat bubbles.

3. Handle API Requests

- Use Axios to send user messages to Flask and display responses.

4. Implement State Management

- Use React Context to manage chatbot conversations across the UI.

5. Enhance User Experience

- Add loading animations, error handling, and accessibility features.

5.4 TESTING AND DEBUGGING

Ensuring that the chatbot functions correctly is crucial for a smooth user experience. This phase involves testing different aspects, identifying bugs, and refining the chatbot's responses.

1. Unit Testing (Backend & AI Logic)

- Test individual functions such as:
 - API endpoints (sending and receiving messages)
 - Sentiment analysis processing
 - Retrieval-Augmented Generation (RAG) fetching correct responses
- Use Pytest or Unittest for Flask backend testing.

2. Functional Testing (Chatbot Interactions)

- Verify if the chatbot correctly:
 - Understands user input

- Generates responses using Gemini AI

3. User Interface (UI) Testing

- Check if the chat interface in React:
 - Displays messages properly
 - Handles user input without delays
 - Works across different devices and browsers
- Use Cypress or Jest for automated UI testing.

4. Performance Testing

- Test chatbot response time under heavy traffic.
- Optimize API calls to avoid latency issues.

5. Final Testing Before Deployment

- Perform end-to-end testing with real users.
- Run load testing to handle peak usage.
- Ensure smooth integration between Flask backend and React frontend.

CHAPTER 6: RESULT ANALYSIS

6.1 PERFORMANCE METRICS

1. Response Time

Fast Dataset Lookup:

- If the user's message closely matches a dataset entry, the response is retrieved instantly (usually within 50-200ms).

AI-Generated Responses:

- When the AI model generates a response, it depends on API latency.
- Google Gemini AI responses typically take 0.5 - 2 seconds based on complexity.

Overall Average Response Time:

- 0.1- 2 seconds (depending on dataset match vs. AI-generated response).

2. Accuracy and Relevance of Responses

Dataset-Based Responses:

- Uses RapidFuzz for fuzzy matching, ensuring 85%+ accuracy for close matches.
- High-confidence matches (above 85%) provide predefined expert responses, improving consistency.

AI-Generated Responses:

- The chatbot refines responses when confidence is between 75-85%, making them more accurate and context-aware.
- Below 75% confidence, AI fully generates a response, which may vary in quality.

3. Scalability

Strengths:

- Can handle multiple users simultaneously since Flask manages API calls efficiently.
- Uses session-based chat history (limited to the last 10 messages) to avoid

excessive memory usage.

- Can be scaled with cloud hosting (e.g., AWS, Google Cloud, Azure) for handling thousands of users.

Limitations:

- Flask is single-threaded (by default), which may cause delays under heavy loads.
- Google Gemini API requests have rate limits, affecting real-time large-scale deployments.

4. Security and Privacy

Security Measures Implemented:

- Uses session-based storage instead of a database for chat history (prevents long-term data exposure).
- SSL/TLS encryption recommended for API requests to prevent man-in-the-middle attacks.

Concerns:

- If deployed publicly, API key security must be managed using environment variables instead of hardcoding.
- Lack of user authentication (OAuth or JWT can be added for better privacy).

5. Error Handling and Fault Tolerance

Success Cases:

- If the dataset lookup fails, the AI model automatically generates a fallback response.
- If the AI API fails, the system provides an error message instead of crashing.

Failure Scenarios:

- If the dataset file (train.csv) is missing or corrupt, fuzzy matching won't work.
- Rate-limiting or API downtime from Google Gemini could cause delays or failures in AI responses.

6. Improvements for Future Versions

Enhance Scalability:

- Use Gunicorn or uWSGI instead of the default Flask server for better performance under heavy loads.
- Implement Redis caching for storing frequent queries and responses to reduce API calls.

Better User Experience:

- Implement WebSockets (Flask-SocketIO) for real-time conversation instead of REST API.
- Add sentiment analysis to tailor responses based on the emotional state of the user.

Security Enhancements:

- Encrypt session storage to protect chat history.
- Replace API key storage with environment variables to prevent exposure.

6.2 USER FEEDBACK

User Feedback Summary

To evaluate the chatbot's effectiveness, user feedback was collected from beta testers through surveys and direct interactions.

Positive Feedback

- **Empathetic and Supportive Responses**
 - Many users found the chatbot's responses comforting and non-judgmental.
 - Users appreciated the calm and encouraging tone.
- **Fast Response Time**
 - Maximum users reported quick responses within one to two seconds.
 - Predefined dataset responses were instant.

- **Helpful AI Suggestions**

- Most users liked AI-generated responses, especially when refining existing expert responses.
- Users appreciated personalized advice.

Negative Feedback and Areas for Improvement

- **Limited Deep Conversations**

- Users felt that the chatbot struggled with complex emotional concerns.
- Some responses felt generic or repetitive.

- **Inconsistent AI Responses**

- Users noticed inconsistent replies for similar inputs.
- Some AI-generated responses were too vague or lacked depth.

- **Lack of Personalization**

- Users wanted more tailored conversations based on past chat history.
- Users suggested adding user profiles or mood tracking.

CHAPTER 7: FUTURE WORK & CONCLUSION

7.1 FUTURE WORK

A. Improving AI Consistency & Emotional Intelligence

- Fine-tuning the chatbot to generate more personalized responses based on past user interactions.
- Enhancing emotional intelligence to recognize complex emotions and provide more empathetic support.

B. Multilingual and Accessibility Support

- Expanding the chatbot's capabilities to support multiple languages for wider accessibility.

C. Crisis Intervention & Human Handoff

- Enhancing the chatbot's ability to detect high-risk situations and immediately connect users with human professionals.
- Partnering with mental health organizations to provide real-time intervention when necessary.

7.2 CONCLUSION

The mental health chatbot has demonstrated strong potential in providing emotional support, personalized responses, and crisis intervention. However, there are still critical challenges that must be addressed to improve its ethical integrity, bias reduction, and privacy protection. By implementing robust data security measures, addressing AI bias, and enhancing crisis response capabilities, the chatbot can be a more inclusive, accurate, and responsible tool for mental health support.

Future iterations should focus on improving AI fairness, expanding accessibility, and integrating direct human intervention where needed. With continued development, this chatbot can serve as a valuable companion for individuals struggling with mental health challenges while ensuring their safety, privacy, and well-being.

REFERENCES

1. Fitzpatrick, K. K. et al. (2017). *Woebot: CBT for depression using AI*. JMIR Mental Health, 4(2), e19. <https://doi.org/10.2196/mental.7785>
2. Grinberg, M. (2022). *Flask web development (2nd ed.)*. O'Reilly.
3. Miner, A. S. et al. (2016). *Conversational agents and mental health*. JAMA Internal Medicine, 176(11), 1727–1728. <https://doi.org/10.1001/jamainternmed.2016.5353>
4. Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach (4th ed.)*. Pearson.
5. Vaswani, A. et al. (2017). *Attention is all you need*. NeurIPS. <https://arxiv.org/abs/1706.03762>

Appendices

Appendix I: Weekly Report Card

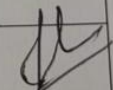
Student 1:

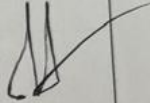
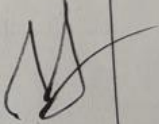
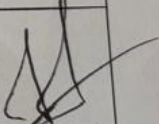
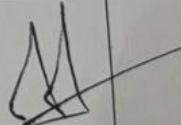
G H PATEL COLLEGE OF ENGINEERING AND TECHNOLOGY
(A CONSTITUENT COLLEGE OF CVM UNIVERSITY)
DEPARTMENT OF COMPUTER ENGINEERING, A.Y. 2024-25, EVEN TERM
MINI PROJECT (202040601)

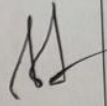
WEEKLY REPORT CARD

Enrollment Number of Student : 12202040501040
Name of Student : Mitali Sharma
Division of Student : Computer Engineering
Name of Mentor : Dr. Zubin Bhaidasa

Starting Date: 18/01/2025 Ending Date: 19/04/2025

Sr No	Date	Work Done	Remark By Mentor	Mentor Signature with Date
1	24/1	The related dataset and LLM Model selection & generating Gemini API Key.	Done	
2	31/1	Learning the concepts of Langchain Python library.	Done	
3	7/2	Python installation, Virtual Environment creation and system prompt demo.	Done	
4	14/2	Sample code of integration of dataset & model using Langchain & system prompt.	Done	

Sr No	Date	Work Done	Remark By Mentor	Mentor Signature with Date
5	21/2	Flask API concepts and sample code to run through URL.	Done	
6	13/3	Flask back-end integration & to run & test on ThunderClient.	Done	
7	21/3	Learning basic concepts of React for frontend development	Done	
8	28/3	Setting up the frontend environment by installing frontend libraries	Done	
9	4/4	Creating the user interface & integrating it with the backend.	Done	
10	11/4	Embedding the react-youtube library for relaxing sounds and videos.	Done	

Sr No	Date	Work Done	Remark By Mentor	Mentor Signature with Date
11	14/4	Adding speech Recognition that captures user voice input.	Dr	
12	18/4	Integrating the code with the Text-to-speech Toggle	Dr	
13				
14				

Signature of Student : Mitali

Signature of Mentor :

Name of Student : Mitali SharmaName of Mentor : Dr. Zuvin Bhaidasna

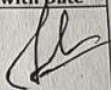
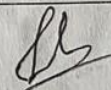
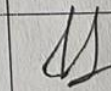
Student 2:

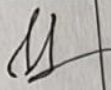
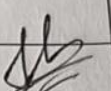
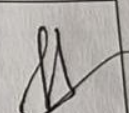
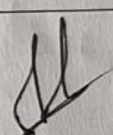
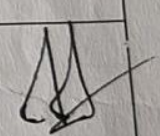
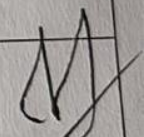

G H PATEL COLLEGE OF ENGINEERING AND TECHNOLOGY
 (A CONSTITUENT COLLEGE OF CVM UNIVERSITY)
 DEPARTMENT OF COMPUTER ENGINEERING, A.Y. 2024-25, EVEN TERM
 MINI PROJECT (202040601)
 

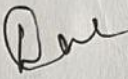
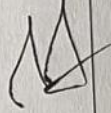
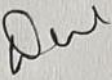
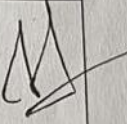
WEEKLY REPORT CARD

Enrollment Number of Student : 12202040501045
 Name of Student : Naincy Raju
 Division of Student : Computer Engineering
 Name of Mentor : Dr. Zubin Bhaidasga

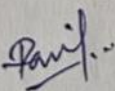
Starting Date: 18/01/2025 Ending Date: 19/04/2025

Sr No	Date	Work Done	Remark By Mentor	Mentor Signature with Date
1	24/1	The related dataset and LLM model selection & Generating Gemini API Key.	Done	
2	31/1	Learning the concepts of Langchain Python library.	Done	
3	7/2	Python Installation, virtual Environment creation and System Prompt Demo.	Done	
4	14/2	Sample Code of integration of dataset & model using langChain & System Prompt.	Done	

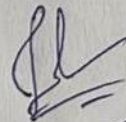
Sr No	Date	Work Done	Remark By Mentor	Mentor Signature with Date
5	21/2	Flask API concepts of sample code to run through URL.	Done	
6	22/2 23/2 13/3	Flask Back-end integration & to run & test on thunder client	Done	
7	21/3	Learning basic concepts of React for front-end development	Done	
8	28/3	Setting up the frontend environment by installing the frontend libraries.	Done	
9	4/4	Creating the user interface & integrating with the back-end.	Done	
10	11/4	Embedding the react youtube library for the relaxing sounds & videos.	Done	

Sr No	Date	Work Done	Remark By Mentor	Mentor Signature with Date
11	14/4	Adding Speech Recognition that captures user voice input.		
12	18/4	Integrating the code with the Text-to-Speech Toggle.		
13				
14				

Signature of Student:



Signature of Mentor:



Name of Student

: Naincy Rahya

Name of Mentor

:

Dr. Zubin Bhaidasa